C PROGRAMMING – ASSIGNMENT ANSWERS

Q4, Q5 and Q6

Name: SAMYUKTHA.R

Register Number: 1913260047

Course: CSA0209

Slot: B

GITHUB RESPIRATORY : samyuktha28-sys/C-programming--CSA0209

Q4 – else–if Ladder or switch? (6 Marks)

The examination cell converts marks into grades: S (90–100), A (80–89), B (70–79), C (60–69), D (50–59) and F (below 50). Marks outside 0–100 are invalid.

1. Implementation using an else–if ladder

```
#include <stdio.h>

char grade_if(int marks)
{
    if (marks < 0 || marks > 100) return '?';
    else if (marks >= 90) return 'S';
    else if (marks >= 80) return 'A';
    else if (marks >= 70) return 'B';
    else if (marks >= 60) return 'C';
    else if (marks >= 50) return 'D';
    else return 'F';
}

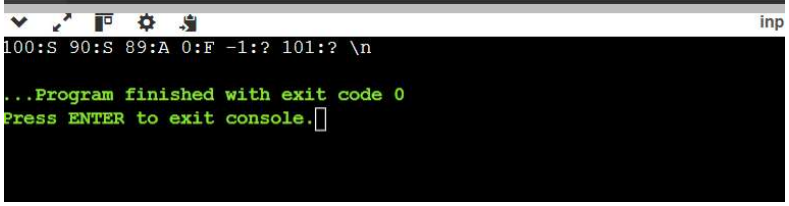
int main(void)
{
    int tests[] = {100, 90, 89, 0, -1, 101};
    for (int i = 0; i < 6; i++)
        printf("%d:%c ", tests[i], grade_if(tests[i]));
    printf("\\n");
    return 0;
}
```

Output: 100:S 90:S 89:A 0:F -1:? 101:?

```c
#include <stdio.h>

char grade_if(int marks)
{
    if (marks < 0 || marks > 100) return '?';
    else if (marks >= 90) return 'S';
    else if (marks >= 80) return 'A';
    else if (marks >= 70) return 'B';
    else if (marks >= 60) return 'C';
    else if (marks >= 50) return 'D';
    else return 'F';
}

int main(void)
{
    int tests[] = {100, 90, 89, 0, -1, 101};
    for (int i = 0; i < 6; i++)
        printf("%d:%c ", tests[i], grade_if(tests[i]));
    printf("\\n");
    return 0;
}
```



2. Completed implementation using switch

```c
#include <stdio.h>

char grade(int marks)
{
    if (marks < 0 || marks > 100)
        return '?';

    switch (marks / 10)
    {
        case 10:
        case 9:  return 'S';
        case 8:  return 'A';
        case 7:  return 'B';
        case 6:  return 'C';
        case 5:  return 'D';
        default: return 'F';
    }
}

int main(void)
{
    int marks[] = {100, 95, 83, 50, 49, -5};
    for (int i = 0; i < 6; i++)
        printf("%d:%c ", marks[i], grade(marks[i]));
    printf("\\n");
    return 0;
}
```

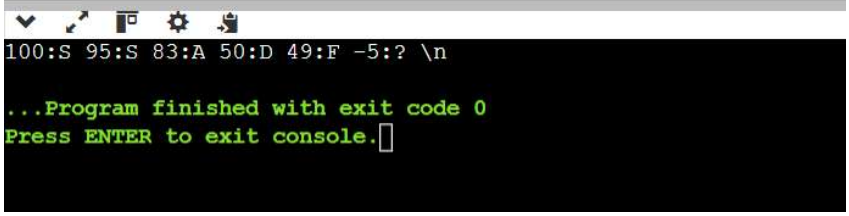
Expected output: 100:S 95:S 83:A 50:D 49:F -5:?

```c
#include <stdio.h>

char grade(int marks)
{
    if (marks < 0 || marks > 100)
        return '?';

    switch (marks / 10)
    {
        case 10:
        case 9:  return 'S';
        case 8:  return 'A';
        case 7:  return 'B';
        case 6:  return 'C';
        case 5:  return 'D';
        default: return 'F';
    }
}

int main(void)
{
    int marks[] = {100, 95, 83, 50, 49, -5};
    for (int i = 0; i < 6; i++)
        printf("%d:%c ", marks[i], grade(marks[i]));
    printf("\\n");
    return 0;
}
```



```
100:S 95:S 83:A 50:D 49:F -5:? \n

...Program finished with exit code 0
Press ENTER to exit console.
```

3. Comparison and boundary tests

Input marks	Expected	else–if	switch
100	S	S	S
90	S	S	S
89	A	A	A
0	F	F	F
-1	Invalid (?)	?	?
101	Invalid (?)	?	?

Both implementations are correct when validation is done first. The switch uses integer division marks / 10, so 90–99 map to 9 and 100 maps to 10.

A complete boundary test set checks both sides of every grade threshold:

Marks	Expected grade
-1	Invalid (?)
0	F
49	F
50	D
59	D
60	C
69	C
70	B
79	B
80	A

89	A
90	S
99	S
100	S
101	Invalid (?)

4. Missing break and range testing

In a switch, omitting break (or return) allows execution to continue into the next case. This is called fall-through. For example, if case 9 assigns S but has no break, execution can continue into case 8 and overwrite the grade with A. In the completed program, each case returns immediately, so an explicit break is unnecessary.

A switch compares one expression with constant case values. It cannot directly express a range such as marks >= 75 && marks <= 84. An else–if ladder can test relational and compound conditions directly.

5. Which structure is easier to change?

An else–if ladder is generally easier when grade boundaries change to irregular ranges, because the exact range conditions can be edited directly. A switch is concise when categories map neatly to integer values such as marks / 10.

6. Scholarship classification design decision

A nested if is the clearest choice when scholarship eligibility depends on both grade and family-income bands. First test the grade, then test the income band within that grade. A switch inside an if is possible for grade categories, but income ranges still need if conditions. A lookup table can be efficient when all grade/income combinations are fixed and numerous, but it is less transparent for a small set of rules.

```
if (grade == 'S') {
    if (income < 200000)
        printf("Full Scholarship");
    else
        printf("Partial Scholarship");
} else if (grade == 'A') {
    if (income < 200000)
        printf("Partial Scholarship");
    else
        printf("No Scholarship");
} else {
    printf("No Scholarship");
}
```

The income thresholds and scholarship outcomes above are illustrative; the university's actual policy must supply the official values.

7. Lookup-table alternative and complexity

```
char grades[11] = {'F','F','F','F','F','D','C','B','A','S','S'};

char grade_table(int marks)
{
    if (marks < 0 || marks > 100) return '?';
    return grades[marks / 10];
}
```

Method	Time per grade	Extra space
else–if ladder	Up to k condition checks; O(k)	O(1)
switch	O(1) for this fixed classification	O(1)
Lookup table	O(1) array access	O(1), 11 characters

The lookup table is fast and compact, but assumes the grade boundaries align with marks / 10. The ladder is more flexible for irregular boundaries.

Q4 conclusion: Both the else–if ladder and switch can produce correct grades when input validation is included. Switch is concise for fixed categories, while else–if is more flexible for ranges. Nested if is the clearest design for scholarship decisions involving both grade and income.

Q5 – Count the Work Done by a Loop (7 Marks)

A number is prime if it is greater than 1 and has no divisor other than 1 and itself. If n has a divisor larger than √n, it must have a corresponding divisor smaller than √n. Therefore, testing divisors only up to √n is sufficient.

1. Improved for-loop implementation (completed TODO)

```
#include <stdio.h>

unsigned long checks = 0;

int is_prime(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;

    for (unsigned long i = 2; i <= n / i; i++)
    {
        checks++;
        if (n % i == 0)
            return 0;
    }
    return 1;
}

int main(void)
{
    unsigned long tickets[] = {97, 100, 7919, 1000003};

    for (int k = 0; k < 4; k++) {
        int p = is_prime(tickets[k]);
        printf("%8lu %-9s divisions = %lu\\n",
               tickets[k], p ? "prime" : "not prime", checks);
    }
    return 0;
}
```

Expected output:

```
      97 prime     divisions = 8
     100 not prime divisions = 1
    7919 prime     divisions = 87
 1000003 prime     divisions = 999
```

The condition i <= n / i is equivalent to i * i <= n for positive i, but avoids overflow from multiplying i by itself.

```c
#include <stdio.h>

unsigned long checks = 0;

int is_prime(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;

    for (unsigned long i = 2; i <= n / i; i++)
    {
        checks++;
        if (n % i == 0)
            return 0;
    }
    return 1;
}

int main(void)
{
    unsigned long tickets[] = {97, 100, 7919, 1000003};

    for (int k = 0; k < 4; k++) {
        int p = is_prime(tickets[k]);
        printf("%8lu %-9s divisions = %lu\\n",
               tickets[k], p ? "prime" : "not prime", checks);
    }
    return 0;
}
```

```
input
      97 prime     divisions = 8\n     100 not prime divisions = 1\n     7919 prime     divisions = 87\n 1000003 prime     divisions = 9
99\n

...Program finished with exit code 0
Press ENTER to exit console.
```

2. Original for loop: test every possible divisor

```c
int is_prime_original(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;

    for (unsigned long i = 2; i < n; i++) {
        checks++;
        if (n % i == 0) return 0;
    }
    return 1;
}
```

3. Improved while loop

```c
int is_prime_while(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;

    unsigned long i = 2;
    while (i <= n / i) {
        checks++;
        if (n % i == 0) return 0;
        i++;
    }
    return 1;
}
```

4. Improved do–while loop

```c
int is_prime_do(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;
```

```
    unsigned long i = 2;
    do {
        if (i > n / i) break;  /* check before dividing */
        checks++;
        if (n % i == 0) return 0;
        i++;
    } while (1);

    return 1;
}
```

A do–while loop always executes its body at least once. For n = 2 or n = 3, no divisor needs to be tested. The boundary check inside the body must therefore happen before the modulo operation; otherwise an unnecessary division is counted.

5. Division counts and comparison

Ticket	Original loop (2 to n−1)	Improved loop (to √n)
97	95	8
100	1	1
7919	7917	87
1000003	1000001	999

The original loop stops early for composite numbers when it finds a divisor. For prime numbers, it tests all divisors through n − 1. The improved loop checks only through the square root.

If the condition is changed to i < n, the prime-number counts become 95 for 97, 7917 for 7919 and 1000001 for 1000003. For 100, early termination still occurs at divisor 2, so the count remains 1.

6. Why i * i <= n can overflow

For sufficiently large unsigned integers, i * i can exceed the maximum representable value and wrap around. The comparison may then become incorrect. Using i <= n / i avoids the multiplication and is safer for positive n.

7. Sieve of Eratosthenes alternative

```
#include <stdio.h>
#define N 1000003

int main(void)
{
    static unsigned char prime[N + 1];

    for (int i = 0; i <= N; i++)
        prime[i] = 1;

    prime[0] = prime[1] = 0;

    for (int i = 2; i <= N / i; i++) {
        if (prime[i]) {
            for (int j = i * i; j <= N; j += i)
                prime[j] = 0;
        }
    }

    printf("97: %s\\n", prime[97] ? "prime" : "not prime");
    printf("100: %s\\n", prime[100] ? "prime" : "not prime");
    printf("1000003: %s\\n",
           prime[1000003] ? "prime" : "not prime");
    return 0;
}
```

```c
#include <stdio.h>
#define N 1000003

int main(void)
{
    static unsigned char prime[N + 1];

    for (int i = 0; i <= N; i++)
        prime[i] = 1;

    prime[0] = prime[1] = 0;

    for (int i = 2; i <= N / i; i++) {
        if (prime[i]) {
            for (int j = i * i; j <= N; j += i)
                prime[j] = 0;
        }
    }

    printf("97: %s\\n", prime[97] ? "prime" : "not prime");
    printf("100: %s\\n", prime[100] ? "prime" : "not prime");
    printf("1000003: %s\\n",
           prime[1000003] ? "prime" : "not prime");
    return 0;
}
```

input

```
97: prime\n100: not prime\n1000003: prime\n

...Program finished with exit code 0
Press ENTER to exit console.
```

Approach	Time complexity	Space complexity
Trial division, original	O(n) per prime query; may stop early for composites	O(1)
Trial division, improved	O(√n) per query	O(1)
Sieve of Eratosthenes	O(N log log N) once; O(1) per later query	O(N)

The sieve is a good choice when many ticket numbers up to a known maximum N must be checked. For only a few isolated ticket numbers, improved trial division avoids allocating an array of N flags.

Q5 conclusion: The square-root boundary greatly reduces work. For-loop, while-loop and do–while can all implement the improved test, but do–while requires an explicit boundary check before its first division. Counting checks provides measurable evidence of the efficiency improvement.

Q6 – Loops That Must Terminate: Loan Repayment Schedule (8 Marks)

A loan balance earns monthly interest, after which a repayment is made. The program must print each month's interest and payment, stop when the balance is cleared, and report total interest and the number of months.

1. Suitable loop and termination condition

A while loop is suitable because the number of months is not known in advance and the loop should continue only while a balance remains. Use a tolerance such as balance > 0.005 rather than balance != 0, because floating-point calculations may leave a tiny rounding residue.

In the final month, if the amount due is less than the regular repayment, pay only the amount due. This avoids overpayment. If the repayment is less than or equal to the interest charged each month, the balance will not decrease; the program must detect this and report that the loan cannot be repaid under the current terms.

2. C program with a 360-month safety limit

```
#include <stdio.h>

int main(void)
{
    double balance, rate, repayment;
    double total_interest = 0.0;
    int month = 0;
    const double EPS = 0.005;
    const int MAX_MONTHS = 360;

    printf("Enter principal, monthly rate (%%), repayment: ");
    if (scanf("%lf %lf %lf", &balance, &rate, &repayment) != 3 ||
        balance <= 0 || rate < 0 || repayment <= 0) {
        printf("Invalid loan details.\\n");
        return 1;
    }

    rate /= 100.0;

    if (repayment <= balance * rate + EPS) {
        printf("Repayment is not greater than monthly interest; "
               "loan will not reduce.\\n");
        return 0;
    }

    printf("\\nMonth  Opening Balance  Interest  Payment  Closing Balance\\n");

    while (balance > EPS) {
        if (month >= MAX_MONTHS) {
            printf("Safety limit reached: loan is not fully repaid.\\n");
            break;
        }

        double opening = balance;
        double interest = opening * rate;
        double due = opening + interest;
        double payment = repayment;

        if (payment > due)
            payment = due;  /* cap payment in the final month */

        balance = due - payment;
        if (balance < EPS)
            balance = 0.0;

        total_interest += interest;
        month++;

        printf("%5d  %15.2f  %8.2f  %8.2f  %15.2f\\n",
               month, opening, interest, payment, balance);
    }

    if (balance <= EPS)
        printf("\\nLoan repaid in %d months. Total interest = Rs %.2f\\n",
               month, total_interest);

    return 0;
}
```

```c
#include <stdio.h>

int main(void)
{
    double balance, rate, repayment;
    double total_interest = 0.0;
    int month = 0;
    const double EPS = 0.005;
    const int MAX_MONTHS = 360;

    printf("Enter principal, monthly rate (%%), repayment: ");
    if (scanf("%lf %lf %lf", &balance, &rate, &repayment) != 3 ||
        balance <= 0 || rate < 0 || repayment <= 0) {
        printf("Invalid loan details.\\n");
        return 1;
    }

    rate /= 100.0;

    if (repayment <= balance * rate + EPS) {
        printf("Repayment is not greater than monthly interest; "
               "loan will not reduce.\\n");
        return 0;
    }

    printf("\\nMonth  Opening Balance  Interest  Payment  Closing Balance\\n");

    while (balance > EPS) {
        if (month >= MAX_MONTHS) {
            printf("Safety limit reached: loan is not fully repaid.\\n");
```

```
input
Enter principal, monthly rate (%), repayment: 10000 2 2000
\nMonth  Opening Balance  Interest  Payment  Closing Balance\n    1         10000.00    200.00   2000.00          8200.00\n    2
 8200.00    164.00   2000.00          6364.00\n    3          6364.00    127.28   2000.00          4491.28\n    4          4491.28     89
.83   2000.00          2581.11\n    5          2581.11     51.62   2000.00          632.73\n    6          632.73     12.65    645.38
         0.00\n\nLoan repaid in 6 months. Total interest = Rs 645.38\n

...Program finished with exit code 0
Press ENTER to exit console.
```

3. Hand-calculated example

Principal = Rs 10,000; monthly interest rate = 2%; regular repayment = Rs 2,000. Interest is calculated on the opening balance each month, then the repayment is deducted. The final payment is capped at the amount due.

Month	Opening balance (Rs)	Interest 2% (Rs)	Payment (Rs)	Closing balance (Rs)
1	10,000.00	200.00	2,000.00	8,200.00
2	8,200.00	164.00	2,000.00	6,364.00
3	6,364.00	127.28	2,000.00	4,491.28
4	4,491.28	89.83	2,000.00	2,581.11
5	2,581.11	51.62	2,000.00	632.73
6	632.73	12.65	645.38	0.00

Total interest = Rs 645.38 (rounded to two decimal places). Total duration = 6 months. The final payment is Rs 645.38, not Rs 2,000, because that is the remaining balance plus the final month's interest.

4. Effect of reducing the repayment

The balance changes approximately according to: new balance = old balance × (1 + r) − E, where r is the monthly interest rate as a decimal and E is the repayment. As E approaches the initial monthly interest rP from above, repayment progress becomes slower and the number of months increases sharply. If E is equal to or below the interest on the current balance, the balance does not decrease at that point. In particular, if E ≤ rP initially, the loan will not amortise under a fixed repayment; the program should reject or warn about this case.

A practical experiment keeps P = Rs 10,000 and r = 2% and tries repayments such as Rs 2,000, Rs 1,000, Rs 500 and Rs 201. As the repayment approaches Rs 200 (the first month's interest), the term becomes very long. A repayment of Rs 200 or less does not reduce the initial balance.

5. Safety limit and deployment decision

The 360-month limit prevents an accidental infinite or impractically long schedule. If the limit is reached while the balance is still positive, the program must stop and clearly report that the loan has not been fully repaid. It must not print a successful repayment message or silently treat the remaining balance as zero.

6. Direct formula alternative and complexity

For a fixed payment E made at the end of each month, the theoretical number of payments is:

$$n = -\log(1 - rP / E) / \log(1 + r)$$

Here P is the principal, r is the monthly interest rate as a decimal, and E is the fixed payment. The formula is valid when E > rP. The result is generally not an integer; take the ceiling to estimate the number of payments, and calculate the exact final payment separately if it is smaller than E.

```
#include <math.h>

/* Returns the theoretical number of payment periods.
   Caller must check E > r * P before calling. */
double months_needed(double P, double r, double E)
{
    return -log(1.0 - r * P / E) / log(1.0 + r);
}

/* Example number of scheduled payments:
   int months = (int)ceil(months_needed(10000, 0.02, 2000)); */
```

Method	Time	Space	Notes
Monthly simulation	O(n) iterations	O(1)	Shows each month, interest and final adjusted payment; floating-point rounding accumulates.
Direct formula	O(1)	O(1)	Fast estimate, but logarithm precision and rounding affect the result; final payment still needs calculation.

For r = 0, the formula needs a separate case: the number of payments is P / E, rounded up. If E ≤ rP for positive r, the logarithm is not valid for a finite payoff and the loan does not amortise.

Q6 conclusion: A while loop best matches a loan schedule because the number of months is unknown. A tolerance-based balance check, final-payment adjustment, repayment-versus-interest validation and 360-month safety limit make the program safer. Simulation is best when a month-by-month record is required, while the formula gives a quick estimate of the duration.

Overall Conclusion

These three problems demonstrate how program structure affects correctness, efficiency and maintainability. In Q4, an else–if ladder is flexible for ranges, while switch and lookup tables are concise for fixed categories; scholarship decisions are best expressed with nested conditions when both grade and income matter. In Q5, restricting divisor checks to the square root reduces prime testing from linear work to O(√n), and a sieve is preferable when many values up to a fixed limit are queried. In Q6, a while loop models an unknown repayment duration, but it must include floating-point tolerance, a final adjusted payment, a non-amortising-loan check and a safety limit. Carefully selected boundary tests and measured operation counts provide evidence that the programs behave correctly.